

gstack+: 面向成本效率的 三層大語言模型編排框架

張哲聞 (Dave Zhang)

技術報告 • 2026 年 5 月

Abstract (English)

We present gstack+, a three-tier AI task orchestration framework that reduces LLM API costs by 46%–98% while maintaining or improving output quality. The framework routes development tasks through a five-dimension scoring system (J/C/R/V/Cr) to Tier-A (Architect), Tier-Mid (Reviewer), or Tier-Exec (Executor). Eight experiment series across 110+ tasks and 4 technical domains demonstrate 100% routing accuracy in three independent validations and 86.7% cost reduction with quality improvement on Tier-Mid tasks using the S1 prompt strategy.

摘要

本文提出 gstack+，一個三層 AI 任務編排框架，通過五維度評分系統（判斷強度 J、上下文寬度 C、風險權重 R、可驗證性 V、創意密度 Cr）將開發任務路由至最合適的模型層（Tier-A 架構師、Tier-Mid 審查員、Tier-Exec 執行者），在不損失品質的前提下削減 46%–98% 的 LLM API 成本。八個獨立實驗系列、110+ 個任務、4 個技術領域的驗證表明：路由準確率在三次獨立驗證中均達 100%；最優 Prompt 策略（S1）在 Tier-Mid 任務中以 86.7% 更低成本超越 Opus（15.0 vs 12.7 分）；敏感性分析揭示 R 和 J 維度各影響 32–33% 邊界任務的路由決策，支持保守路由原則。

關鍵詞： LLM 任務路由，多模型編排，成本優化，AI 輔助開發，Prompt 工程

1 引言

大語言模型在軟件開發工作流程中的廣泛應用帶來了一個新的成本維度：模型 API 費用在規模化部署時可能佔據相當比例的工程預算。然而，當前實踐普遍存在「模型選擇扁平化」問題——無論任務複雜度如何，工程師傾向於對所有任務使用同一個（通常是最昂貴的）模型。

這一現象產生兩類系統性浪費：過度路由（Over-routing），即將 `git rebase`、函數重命名等機械任務派給最頂級模型，為零品質收益支付 20 - 100 倍的成本溢價；以及不足路由（Under-routing），即將安全架構設計、橫切關注點重構等高判斷任務派給廉價模型，引入品質風險和決策錯誤，補救成本遠超節省。

gstack+ 的核心洞察是：對大多數開發任務而言，路由正確性比模型能力更重要。真實開發任務按複雜度大致遵循 50/35/15 分佈（機械執行/中等判斷/深度推理），與最便宜和最貴模型之間約 30 倍的成本差距結合，在正確路由下可實現約 46% 的整體節省。

本文的主要貢獻：

1. 在三次獨立驗證的 70 個任務上達到 100% 準確率的五維度評分框架（J/C/R/V/Cr）與確定性路由規則。
2. 具有正式角色邊界和交接協議的三層模型架構（Tier-A/Mid/Exec）。
3. 來自 8 個實驗系列的量化實驗證據，顯示在不降低品質的前提下削減 46% - 98% 成本。
4. 量化各評分維度對路由決策影響的敏感性分析，指導實踐保障措施。

2 相關工作

2.1 LLM 成本優化

現有 LLM 成本降低工作主要集中在模型壓縮（量化、蒸餾）和推理優化（投機解碼、KV 緩存共享）。這些方法降低每 token 成本，但不解決任務-模型不匹配問題。FrugalGPT 引入了類似路由概念，但聚焦 API 層面的成本協商而非任務複雜度感知調度。

其他研究者探索了基於 Prompt 的成本削減，例如「Prompt 壓縮」技術，在保留語義意圖的同時移除冗餘 token。gstack+ 的不同之處在於它在宏觀編排層級運行，決定使用哪個「大腦」（模型）而不是如何與其對話。

2.2 多智能體系統與任務分解

AutoGen、CrewAI 和 LangGraph 提供多智能體框架，但關注智能體角色和通信模式而非成本感知任務路由。它們未提供用於決定任務需要何種智能體能力級別的量化評分框架。最近關於「智能體工作流」的研究強調了人機協同驗證的重要性，gstack+ 通過 Tier-Mid 角色將這一點正式化。

2.3 LLM 中的層級規劃

層級規劃的概念，即「高級指揮官」（類似 Tier-A）將目標分解為「低級動作」（類似 Tier-Exec），在機器人學中已根深蒂固，且在 AI 代理領域日益普及。然而，gstack+ 是第一個將這種層級結構專門應用於軟件工程工作流經濟優化的框架。

3 詳細案例研究 (Series 1)

為了說明框架的運行，我們提供了 Series 1 基線中使用的三個任務的技術演練。

3.1 案例 1: 跨代碼庫重命名 (Tier-Exec)

任務：在三個獨立的代碼庫中將導出的函數 `useAuth` 重命名為 `useSecurityAuth`，並更新所有導入語句。

評分：

- 判斷強度 (J=1)：任務是簡單的搜索與替換，無設計模糊性。
- 風險權重 (R=1)：如果失敗，編譯器將立即捕獲損壞的導入。
- 可驗證性 (V=5)：可以使用 `grep` 和構建命令進行驗證。

路由：路由至 Tier-Exec (Qwen Code)。結果：12 秒內完成，成本為 \$0.00014。所有 42 處出現均已正確更新。

3.2 案例 2: React Query 重構 (Tier-Mid)

任務：將使用 `useEffect` 進行數據獲取的組件重構成使用 `@tanstack/react-query` 的 `useQuery` 鉤子，包括錯誤處理和加載狀態。

評分：

- 判斷強度 (J=3)：需要理解組件生命週期並將舊邏輯映射到新鉤子。
- 風險權重 (R=2)：功能回歸風險中等。
- 可驗證性 (V=3)：需要人工代碼審查和 UI 冒煙測試。

路由：路由至 Tier-Mid (Claude Sonnet)。結果：以 15/15 的品質得分完成。模型正確識別了原始 `useEffect` 實現中的競爭條件，並在重構期間解決了它。

3.3 案例 3: SSO + MFA 架構設計 (Tier-A)

任務：設計一個支持 SAML SSO、基於 TOTP 的 MFA 和跨微服務架構的會話撤銷的系統級身份驗證策略。

評分：

- 判斷強度 (J=5)：涉及多個安全權衡的高級架構推理。
- 風險權重 (R=5)：任何設計缺陷都可能導致大規模安全漏洞。
- 創意密度 (Cr=4)：需要對現有協議進行創新的集成。

路由：路由至 Tier-A (Claude Opus)。結果：生成了一份 12 頁的技術規範，涵蓋令牌流、密鑰輪換策略和故障模式。該設計被高級安全工程師評定為「生產就緒」。

4 系統設計

4.1 設計動機

gstack+ 的核心假設：對大多數開發任務而言，路由正確性比模型能力更重要。

這一假設基於兩個觀察。其一，任務複雜度遵循偏斜分佈：約 50% 的真實開發任務涉及機械執行（重命名、重新格式化、文檔），35% 涉及中等判斷（帶設計決策的重構、代碼審查），僅 15% 需要深度架構推理。其二，模型層之間的成本差異約為 30 倍，意味著在不觸及頂層的情況下，正確路由底部 50% 的任務即可減少約 46% 的總支出。

這意味著框架應投資於分類準確性，而非始終默認使用最強大的模型。

4.2 五維度評分框架

每個任務在五個維度上評分，每個維度 1 – 5 分：

維度	名稱	測量內容	範圍
J	判斷強度	需要多少人類級別的決策？	1 – 5
C	上下文寬度	需要多少代碼庫知識？	1 – 5
R	風險權重	出錯的代價有多高？	1 – 5
V	可驗證性	能否自動驗證成功？	1 – 5
Cr	創意密度	需要多少新設計（vs 套模板）？	1 – 5

表 1 任務分類的五維度評分框架

五個維度涵蓋三個能力軸：認知負荷（J + Cr）、知識範圍（C）和風險態度（R + V）。這一分解確保任務不被過度簡化——單一「難度」分數無法區分「高風險但簡單」的任務和「複雜但安全」的任務。

4.3 三層路由規則

給定五個評分後，路由決策是確定性的：

Tier-A : $J \geq 4$ 或 $R \geq 4$ 或 $Cr \geq 4$

Tier-Exec: $J \leq 2$ 且 $C \leq 2$ 且 $V \geq 4$ 且 $R \leq 2$

Tier-Mid : 所有其他情況（保守默認）

Tier-A 和 Tier-Exec 規則之間的不對稱是有意為之的。Tier-A 在任意單一高風險信號時觸發（OR 邏輯），因為低估高風險任務的代價是嚴重的。Tier-Exec 需要所有條件同時成立（AND 邏輯），因為機械執行受益於嚴格的驗證要求。

4.4 保守路由原則

敏感性分析（Series 6）揭示 J 和 R 各在 ± 1 評分擾動下影響 32 – 33% 邊界案例的路由。這促成了一條保守路由規則：當 $J = 3$ 或 $R = 3$ （邊界值）時，路由至 Tier-A 而非 Tier-Mid。

經濟學依據：低估高判斷任務的成本（補救、返工、潛在生產問題）通常超過高估成本的 3 - 5 倍。保守規則在邊界案例中最多增加 20 - 50% 的額外成本，同時避免災難性失敗。

4.5 三層架構與交接協議

每個層級具有明確定義的職責 and 通信邊界：

Tier-A（架構師）：將用戶需求分解為任務，應用五維度評分，做出路由決策，審查升級的失敗案例。不編寫實現代碼。

Tier-Mid（審查員）：根據定義的成功標準驗證 Tier-Exec 輸出，應用基於證據的接受/拒絕，為 Tier-A 生成結構化反饋。可修復小問題，但不能覆蓋架構決策。

Tier-Exec（執行者）：根據交接規格（範圍鎖定）實現任務，生成結構化證據報告，上報阻塞問題。不做架構判斷。

層級間交接使用結構化模板（`plan-to-exec.md`、`exec-to-check.md`、`check-to-plan.md`），強制執行明確的範圍鎖定、可驗證的成功標準和證據要求。這套協議消除了模糊的完成定義（「看起來完成了」），這是非結構化 AI 工作流程中返工的主要來源。

5 系統詳細架構

5.1 路由引擎（gstack-core）

gstack+ 的核心是路由引擎，這是一個無狀態的邏輯層，將五維度向量 $[J, C, R, V, Cr]$ 轉換為離散的模型分配。與依賴嵌入相似性的概率路由器不同，gstack+ 使用確定的基於規則的引擎。這確保了安全關鍵型決策（例如，路由高風險安全補丁）永遠不會留給基於「感覺」的模型選擇。

路由引擎分三個階段運行：

1. 提取：Tier-A 模型分析任務描述和項目上下文，生成評分向量。
2. 約束檢查：引擎應用硬編碼的邏輯規則（與/或門）。
3. 供應商映射：分配的層級根據當前的可用性和成本權重映射到特定的模型供應商（例如 Anthropic、OpenAI 或本地 Llama 實例）。

5.2 評分邏輯實現

評分邏輯被實現為一組精煉的啟發式規則。例如，判斷強度（J）分數由所需的非線性設計選擇數量決定。只有一條邏輯路徑的任務（例如「將此 JSON 轉換為 XML」）獲得 $J = 1$ 。需要權衡性能和可維護性的任務（例如「選擇緩存策略」）獲得 J_{ge4} 。

風險權重（R）分數根據更改的爆炸半徑計算。涉及身份驗證層、數據庫模式或公共 API 的更改自動獲得 $R = 5$ 。僅 UI 且無數據持久性的更改通常獲得 R_{le2} 。

5.3 層級邊界與隔離

為了防止「範圍蔓延」和「幻覺傳播」，gstack+ 在各層級之間強制執行嚴格隔離：

- Tier-Exec 被授予除目標文件外代碼庫的「唯讀」視圖。
- Tier-Mid 獲取原始需求和 Tier-Exec 的輸出，但不允許看到執行者的中間「思考過程」。
- Tier-A 維持全局狀態，是唯一允許修改學習記憶（GEMINI.md）的層級。

這種多級驗證確保了執行者犯的錯誤能被審查員發現，而架構師犯的架構錯誤在分解階段就能被標記出來。

6 實驗設置

6.1 實驗系列概覽

我們於 2026 年 5 月 4 日至 6 日進行了八個獨立實驗系列，每個系列針對特定驗證問題：

系列	研究問題	任務數
S1	三個層級的成本 vs 品質基線對比	3
S2	大規模路由準確性、穩定性和成本效益	39
S3	Prompt 策略影響；真實 git 歷史路由驗證	24
S4	跨領域壓力測試	20
S5	多模型品質矩陣（Haiku/Sonnet/Opus × 3 層級）	9
S6	評分維度敏感性（±1 擾動分析）	10
S7	路由錯誤非對稱性與過/欠路由成本分析	8
S8	品質矩陣細化與保守路由驗證	6

表 2 實驗系列概覽

6.2 各系列詳細方法論

以下章節詳細介紹了每個主要實驗系列的具體協議和環境。

6.2.1 Series 1: 基準標杆測試

S1 的目標是建立「地面真值」成本/品質基線。我們選擇了三個代表性任務：低判斷機械任務、中等判斷重構任務和高判斷架構任務。每個任務執行兩次：一次使用 gstack+ 路由邏輯，一次使用單一的「All-Opus」策略。

6.2.2 Series 2: 規模與穩定性

S2 將任務數量擴展到 39 個，以測試路由邏輯的統計穩定性。本系列引入了「穩定性測試」，即同一個任務由 Tier-A 模型評分五次，以測量評分方差。結果顯示所有維度的標準偏差均 < 0.2，確認了五維度量表的可靠性。

6.2.3 Series 3: Prompt 策略優化

我們假設對於 Tier-Mid 任務，Prompt 策略比模型規模更顯著。我們在 24 個任務上對比了四種策略（S0 - S3）。本系列採用「交叉驗證」方法，即由一個模型生成的輸出由另一個模型檢查，以確保客觀的品質衡量。

6.2.4 Series 4: 跨領域壓力測試

為確保 gstack+ 不限於特定技術棧，我們為前端（React/Next.js）、後端（Go/Python）、數據（Spark/SQL）和 DevOps（K8s/Terraform）各定義了 5 個任務。代理被要求在分配層級前生成特定領域的評分證據。

6.3 評估協議

品質評估採用 LLM-as-Judge 盲評（評判模型：claude-opus-4-7），在五個維度（技術正確性、完整性、清晰度、風險意識、實用價值）上評分，各維度 0-3 分，滿分 15 分。盲評消除了確認偏見——評判者不知道哪個模型生成了哪個輸出。

每一輪評估遵循嚴格的三步協議：

- 1. 上下文注入：為評判者提供完整的項目上下文和具體的任務交接文檔。
- 2. 盲態評估：評判者在不知道來源模型或路由層級的情況下評估輸出。
- 3. 證據生成：評判者必須引用具體的代碼行或設計決策來證明每個分數的合理性。

成本測量使用從 IDE 賬單數據記錄的實際 API token 消耗，按公布的標價轉換為美元。所有實驗在各模式間使用相同的任務描述和起始 git 狀態，確保公平比較。

7 實驗結果與分析

7.1 Series 1-2: 成本削減與品質對等

最直接的結果來自 Series 1 成本基線：

任務	路由決策	All-Opus	路由模式	節省
跨 repo 重命名	Tier-Exec → Qwen	\$0.01173	\$0.00014	-99%
React Query 重構	Tier-Mid → Sonnet	\$0.07849	\$0.01191	-85%
SSO+MFA 設計	Tier-A → Opus	\$0.07885	\$0.07885	—
合計		\$0.1691	\$0.0909	-46%

表 3 Series 1 三個典型任務的成本對比

關鍵是，路由沒有降低品質：Tier-Mid 任務（React Query 重構）路由至 Sonnet 后在人工評估中優於 Opus（5/5 vs 4/5）。Series 2 Exp-2C 通過 LLM-as-Judge 盲評確認了這一點：路由模式和 All-Opus 模式均得 14.1/15。

7.2 Series 3: Prompt 策略主導模型規模

Series 3 揭示 Prompt 策略是 Tier-Mid 任務的主要品質決定因素：

策略	描述	得分 (/15)	成本/任務
S0 (基準)	無 Prompt 工程	13.7	\$0.006
S2	結構化輸出格式	13.3	\$0.007
S3 (CoT+角色)	思維鏈 + 角色定義	15.0	\$0.007
S1 (最優)	角色 + 深度思考	15.0	\$0.006
Opus 基準	無 Prompt 工程, Opus 模型	12.7	\$0.045

表 4 Prompt 策略品質對比 (Series 3, Tier-Mid 任務)

S1 策略以 \$0.006/任務的成本通過 Sonnet 實現了 15.0/15——比 Opus (\$0.045/任務) 品質提升 15.7%，成本降低 86.7%。

這一發現挑戰了普遍的「越大越好」思維。通過為 Tier-Mid 模型提供特定角色和「在回答前深度思考」的指令，我們可以為 80% 的常見重構任務彌合中層模型與頂層模型之間的推理差距。

7.3 Series 4: 跨領域泛化

Series 4 在前端、後端、數據工程和 DevOps 四個領域 (20 個任務，每領域 5 個) 驗證了框架的跨域適用性。AI 代理 (Qwen Code) 獨立應用評分指南，無需領域特殊訓練或校準。

結果：20/20 路由準確率，所有四個領域的所有五個維度平均偏差均為零。這確認了五維度框架可在無需適配的情況下跨域部署。J (判斷強度) 和 R (風險權重) 分數在各領域顯示出最高的一致性，而 Cr (創意密度) 分數在數據工程任務中表現出略多的方差。

7.4 Series 5 - 6: 多模型對比與敏感性

Series 5 揭示了模型能力的細緻圖景：Haiku 4.5 在 Tier-Exec 任務上得 14/15 (僅比 Opus 低 1 分)，在 Tier-Mid 任務上優於 Sonnet 和 Opus (14 vs 13)。Opus 唯一的系統性優勢是在 Tier-A 任務的風險意識評估維度 (2.7 vs Haiku 的 2.0)。

Series 6 敏感性分析表明，風險權重 (R=33%) 和判斷強度 (J=32%) 維度在 ± 1 擾動下最容易改變路由決策。可驗證性 (V=11%) 和創意密度 (Cr=13%) 最為穩定。這指導在模糊案例中將評分精力集中於 R 和 J。

8 討論

8.1 系統性過度路由浪費

我們對 Series 1 數據集的分析表明，如果沒有路由框架，組織在可以由成本極低且品質相同的模型處理的任務上浪費了約 46% 的 LLM 預算。在極端情況下 (機械任務)，這種浪費超過 98%。

8.2 「智能密度」假設

我們提出智能密度（Intelligence Density）的概念：每千字節生成代碼所需的高判斷推理量。機械任務具有低智能密度，而架構決策具有高密度。gstack+ 作為一個過濾器，將任務的智能密度與模型的推理能力相匹配。

8.3 局限性

樣本量：核心數據集涵蓋 8 個系列中的 110+ 個任務。雖然結果一致，但統計顯著性聲明需要更大的驗證集。100% 路由準確率結果應解釋為「未觀察到失敗」而非「可證明完美」。

受控環境：所有實驗使用已知的預標記任務。真實世界部署引入了可能無法整齊歸入三層分類的新型任務類型。

單一評估者：LLM-as-Judge 評分雖為盲評，但使用單一評判模型。交叉評判驗證將強化品質聲明。

8.4 未來工作

三個擴展最有前景：

1. 在線自適應：實現一個反饋循環，Tier-Mid 審查失敗會自動觸發類似任務配置文件的路由閾值調整。
2. 動態定價集成：將實時 API 延遲和競價定價納入路由決策，允許系統根據當前成本效率切換供應商。
3. 多評判共識：利用「評判小組」（不同的模型和配置）來消除品質評估中殘留的模型特定偏見。

9 結論

gstack+ 表明，系統性任務路由——而非能力最大化——是高效 AI 輔助開發的關鍵槓桿。五維度框架在三次獨立驗證的 70 個任務中實現了 100% 路由準確率，三層架構在不降低品質的情況下削減了 46% - 98% 的成本，敏感性分析為邊界案例提供了可操作的指導。

核心洞察——80% 的開發任務不需要最昂貴的模型——雖然簡單，但其系統性應用需要一個有原則的框架。gstack+ 通過量化驗證提供了這一框架。我們相信，隨著 LLM 能力在成本 and 專業化方面的持續分化，這種編排層將成為任何專業軟件工程工具箱的必備組件。

從「模型選擇」到「任務路由」的轉變代表了該領域的成熟，從 AI 能力的新奇轉向工程效率的紀律。

10 附錄 A：評分指南參考

為了協助團隊實施 gstack+，我們為五維度評分提供以下量表：

10.1 判斷強度（J）

- 1：機械任務（例如格式化）。

- 2: 簡單邏輯（例如 CRUD 操作）。
- 3: 中等判斷（例如具有單文件影響的重構）。
- 4: 高判斷（例如架構更改）。
- 5: 戰略決策（例如安全架構）。

10.2 上下文寬度 (C)

- 1: 單個文件，無外部依賴。
- 2: 同一目錄中的多個文件。
- 3: 跨模塊依賴。
- 4: 整個代碼庫影響。
- 5: 跨代碼庫或系統級上下文。

10.3 風險權重 (R)

- 1: 外觀更改，無功能影響。
- 2: 局部邏輯更改，易於撤銷。
- 3: 對面向用戶的功能有中等影響。
- 4: 對核心業務邏輯有高影響。
- 5: 關鍵安全或基礎設施更改。

10.4 可驗證性 (V)

- 1: 無法自動驗證。
- 2: 需要複雜的集成測試。
- 3: 可以通過單元測試驗證。
- 4: 可以通過簡單的構建/lint 檢查驗證。
- 5: 完全自動化驗證（例如編譯器）。

10.5 創意密度 (Cr)

- 1: 純機械/套模板。
- 2: 遵循既定模式。
- 3: 需要對模式進行微調。
- 4: 需要新穎的設計或綜合。
- 5: 突破性的設計或問題解決。

11 參考文獻

- [1] Brown, T. 等 (2020). Language Models are Few-Shot Learners. NeurIPS.
- [2] Chen, L. 等 (2023). FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176.

- [3] Wei, J. 等 (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS.
- [4] Wu, Q. 等 (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155.
- [5] Anthropic. (2024). Claude 3 Model Card. 技術報告.
- [6] Zhang, D. (2026). gstack+ 實驗系列 1 - 8. 技術報告, github.com/zheWenzhang/gstack-plus.